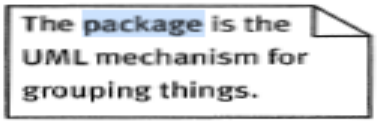
UNIT -III

1. **Packages**

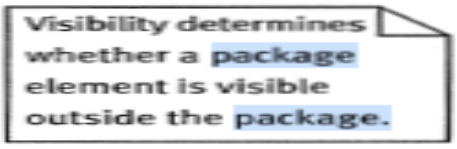
- A package is the general purpose UML mechanism.
- package is a *logical* grouping mechanism.
- To organize model elements (including other packages) and diagrams into groups.
- It is a container.
- Each package has its own namespace – All names must be unique.
- It provides a namespace for its members.
- Elements inside a package can be given a visibility.
- These indicates whether they are visible to clients of the package.

Notation:

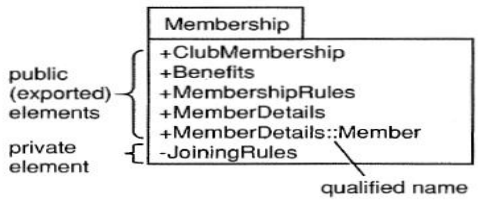
- The package icon is a folder, and
- the package name may be shown on the tab

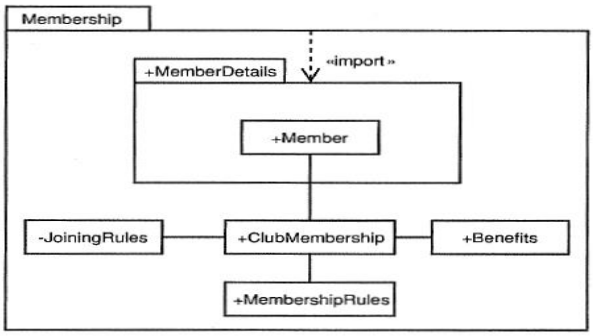


- It can be used
  - To provide an encapsulated namespace within which all names must be unique;
  - To group semantically related elements;
  - To define a "semantic boundary" in the model;
  - To provide units for parallel working and configuration management.



Symbol	Visibility	Semantics
+	public	Elements with public visibility are visible to elements outside the package – they are *exported* by the package
–	private	Elements with private visibility are completely hidden inside the package



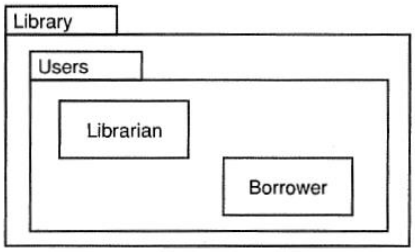


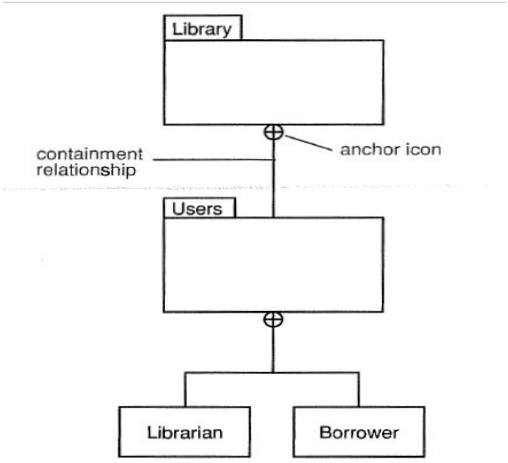
Nested Packages

- This is useful when there is
  - A lot of nesting, or
  - Complex nesting, that be confused to show with embedding.
- Elements in the Users package can access all elements in the Library package by using unqualified names.
- But elements in library must use the qualified names i.e

Users::Librarian & Users::Borrower to access the elements in the users package

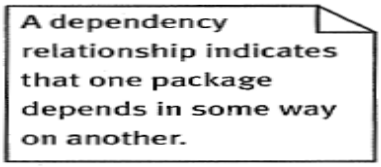
Syntax:



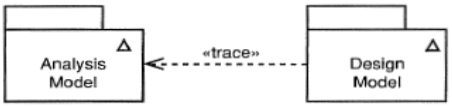


Package Dependencies:

- Any package that has a dependency relationship with the Membership package will be able to see the public elements of that package (ClubMembership, Benefits, etc.)
- But will *not* be able to see the private element JoiningRules.

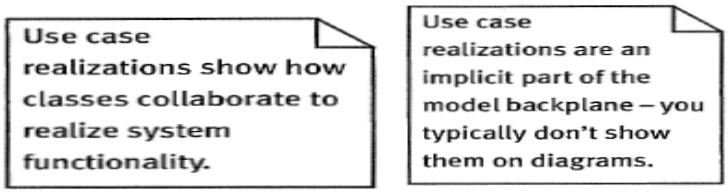


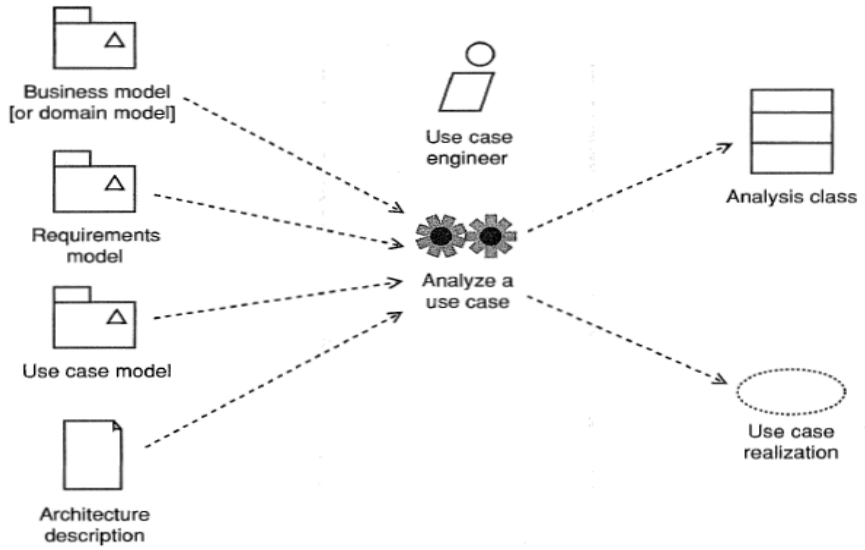
Package dependency types:

	An element in the client package uses a public element in the supplier package in some way – the client depends on the supplier If a package dependency is shown without a stereotype, then «use» should be assumed
	Public elements of the supplier namespace are added as public elements to the client namespace Elements in the client can access all public elements in the supplier using unqualified names
	Public elements of the supplier namespace are added as private elements to the client namespace Elements in the client can access all public elements in the supplier using unqualified names
	«trace» usually represents a historical development of one element into another more developed version – it is usually a relationship between models rather than elements (an extra-model relationship)
	Public elements of the supplier package are merged with elements of the client package This dependency is only used in metamodeling – you should not encounter it in ordinary OO analysis and design

2. Use case Realization

- Use case realization is fundamentally a process of refinement.
  - Analysis classes model the **static structure** of a system.
  - Use case realizations:
    - It shows how instances of the analysis classes interact to the system functionality.
    - It is the **dynamic view** of the system





- The main goals for use case realization in analysis are as follows
  - *Find out* which analysis classes interact to realize the behavior specified by a use case.
  - *Find out* what messages instances of these classes need to send to each other to realize the specified behavior.
  - *Analysis* classes
    - Key operations
    - Key attributes
    - Important relationships between analysis classes
  - *Update* the following models with the information get from use case realization.
    - Use case model,
    - Requirements model, and
    - Analysis classes

Usecase Realization Elements:

Let the use case realizations be an implicit part of the backplane of the model.

The main elements of usecase realization are as follows

1.Analysis class Diagram
2. Interaction Diagrams
3. Special Requirements
4. Use case Refinements

Element	Purpose
Analysis class diagrams	Show the analysis classes that interact to realize the use case
Interaction diagrams	Show interactions between specific instances that realize the use case – they are "snapshots" of the running system
Special requirements	The process of use case realization may well uncover new requirements specific to the use case – these must be captured
Use case refinement	New information may be discovered during realization that means the original use case has to be updated

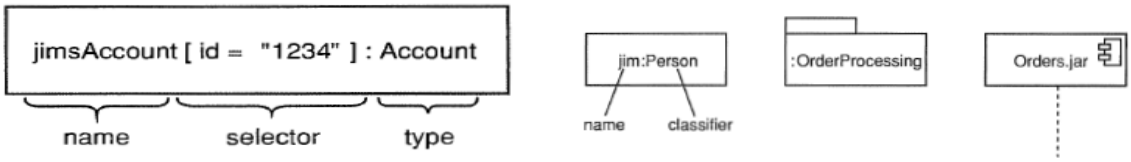
- Interaction diagrams are four types:
  - Sequence diagrams,
  - Communication diagrams,
  - Interaction overview diagrams, and
  - Timing diagrams.
- OO modeling is an iterative process.

3. Interactions:

- It is unit of behavior of a classifier.
- This classifier, known as the *context classifier*.
- *It* provides the context for the interaction.
- An interaction may used as
  - Temporary or Global variable
- In use case realization, the context classifier is a use case.
- The key elements in interaction diagrams are lifelines and messages.

Lifelines:

- A *lifeline* represents a single participant in an interaction.
- It represents how an instance of a specific classifier participates in the interaction.
- Each lifeline has
  - An optional name,
  - A type, and
  - An optional selector



Syntax:

- **Name -** To the lifeline within the interaction.
- **Type -** The name of the classifier of which the lifeline represents an instance.
- **Selector** –
  - A Boolean condition that may be used to select a single instance that satisfies the condition.
  - If there is no selector, a lifeline refers to an arbitrary instance of the classifier.
  - Selectors are only valid

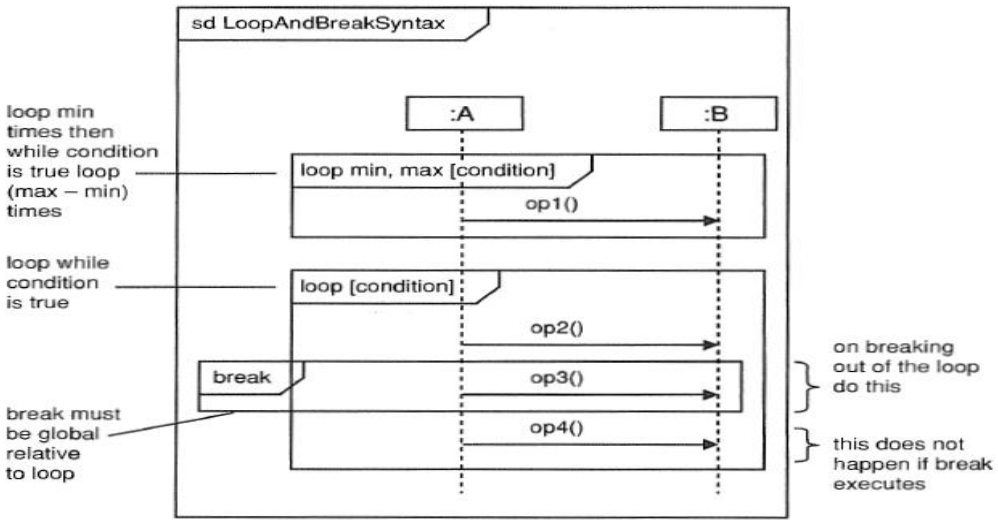
        - If the type has a (multiplicity > 1) so that there are many instances from which to choose.
    - The selector selects an instance of Account that has an id of "1234".
    - Lifelines are drawn with the same icon and
    - Have a vertical dashed "tail" when they are used in sequence diagrams.

Messages:

- Types of messages are
    - Synchronous,
    - Asynchronous, and
    - Return messages
- **synchronous message:**
    - The sender waits for the receiver to finish executing the requested operation.
- **Asynchronous message:**
    - The sender does *not* wait but continues to the next step
- **Return Messages:** Receiver sends response to the senders call

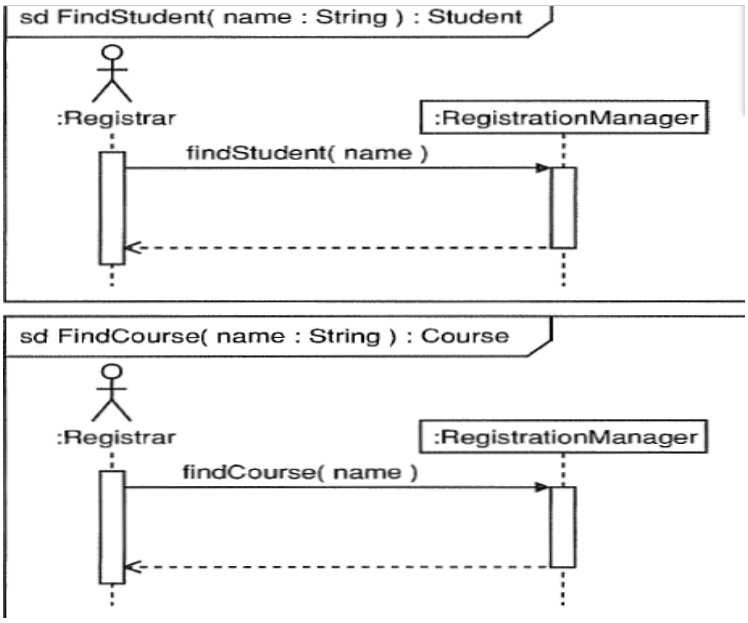
Syntax	Name	Semantics
aMessage(aParameter)	Synchronous message	The sender waits for the receiver to return from executing the message
aMessage(aParameter)	Asynchronous message	The sender sends the message and continues executing – it does *not* wait for a return from the receiver
	Message return	The receiver of an earlier message returns focus of control to the sender of that message
«create» aMessage() :A	Object creation	The sender creates an instance of the classifier specified by the receiver
«destroy»	Object destruction	The sender destroys the receiver If its lifeline has a tail, this is terminated with an X
	Found message	The sender of the message is outside the scope of the interaction Use this when you want to show a message receipt, but don't want to show where it came from
	Lost message	The message never reaches its destination May be used to indicate error conditions in which messages are lost

Loops representations:

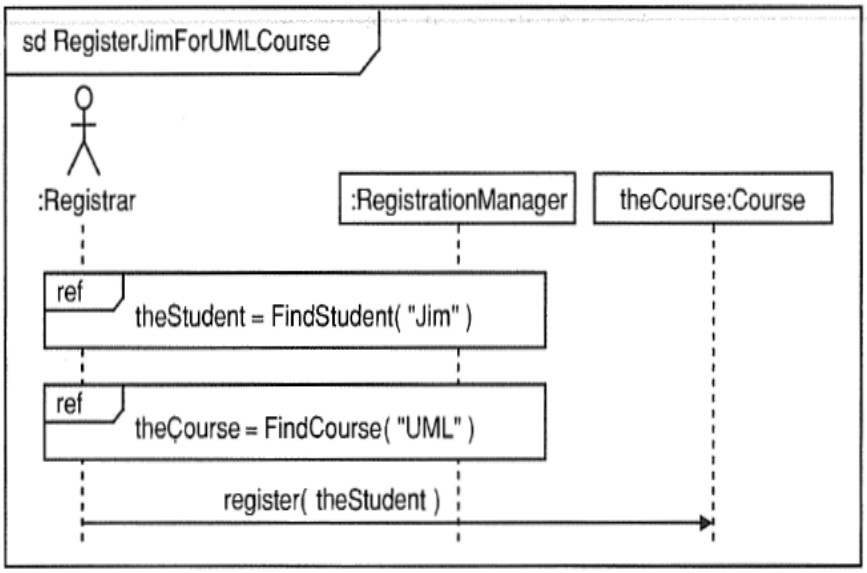


Parameters

- To supply different values to the interaction.
- Let FindStudent(...) and FindCourse(...), two parameterized interactions.

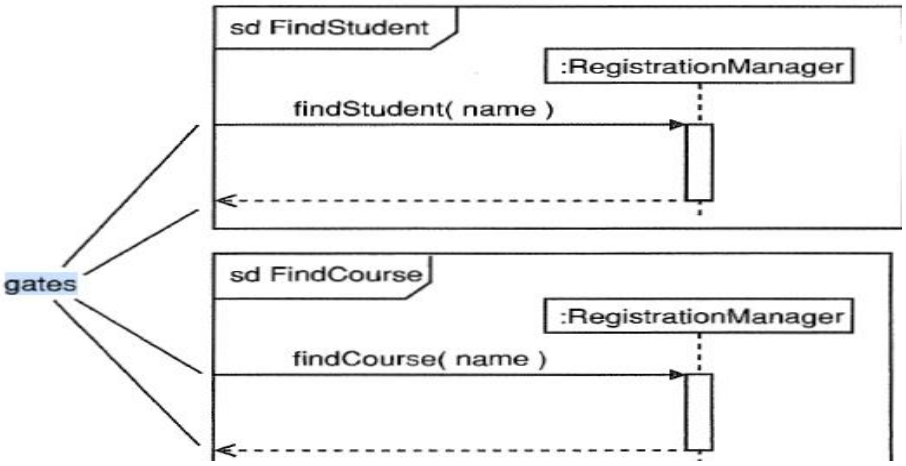


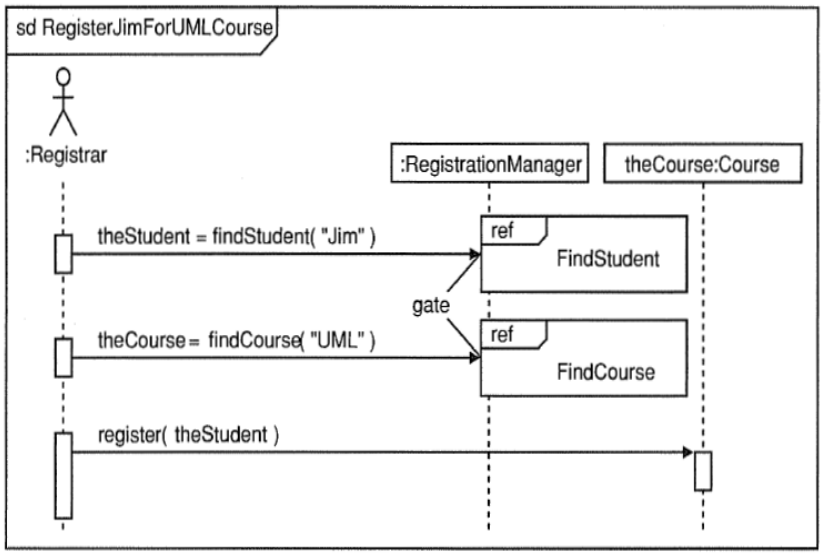
- Pass specific values in to the interactions as parameters.
- The two interaction occurrences have return values that are assigned to the temporary variables theStudent and theCourse.
- These temporary variables exist in the scope of the sequence diagram

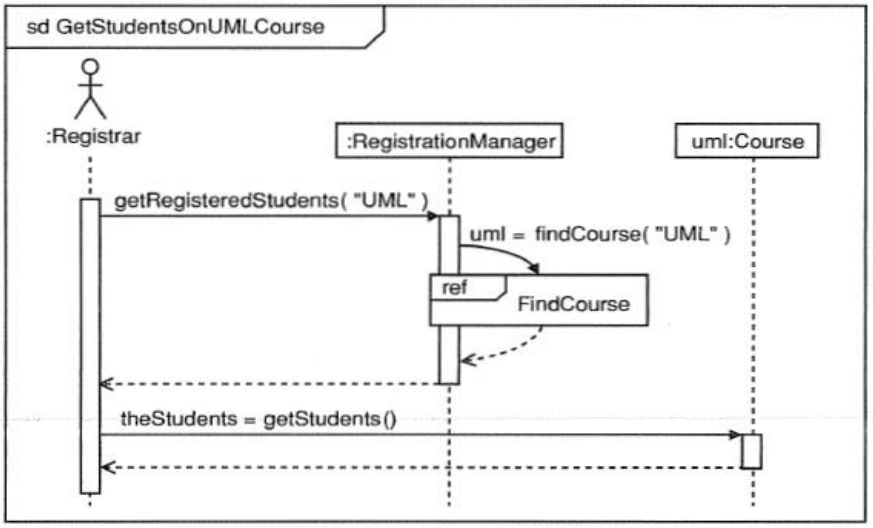


Gates:

- Gates are the inputs and outputs of interactions.
- A gate is a point on the sequence diagram frame.
- Use gates when an interaction to be triggered by a lifeline.
- That is *not* part of the interaction
- This point connects a message *outside* the frame to a message *inside* the frame.
- Both these messages *must* have a matching signature.
- New sequence diagrams with gates are as follows
- FindStudent and FindCourse have explicit inputs and outputs.



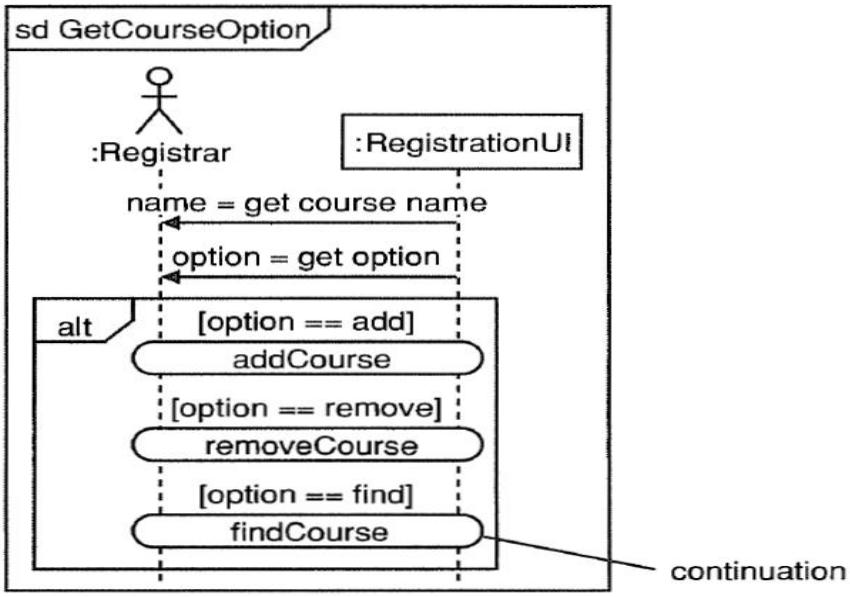




Continuation

- Continuations provide a way to connect different interactions.
- **Ssyntax**
  - It is drawn as a label inside a rounded rectangle
- Especially one interaction finishes, **leaving** its lifelines in a specific state, and other interactions may **pick up at that point** and continue.

- Continuations allow an interaction fragment
- To indicate that its flow terminates in such a way that it can be picked up and continued by another interaction fragment.
- When (Continuation = = first item) – indicates the fragment is continuing from previous fragment.
- When (Continuation = = Last item) – indicates the point at which fragment finishes.
- Note: Communication diagrams emphasize the structural aspects of an interaction.



4. **Activity Diagram:**

- Activity diagrams are OO flowcharts.
- Activity refer to features of its context
- To model a process as an activity that consists of a collection of nodes connected by edges.
- In UML 1, activity diagrams were special cases of state machines.
  - Every state had an entry action.
  - It specified some process or function that occurred when the state was entered.
- In UML 2, activity diagrams have completely new semantics based on Petri Nets.
- It has two advantages
  - The Petri Net formalism provides greater flexibility in modeling different types of flow.
  - It shows a clear distinction between activity diagrams and state machines.
- Activities are typically attached to
  - use cases;
  - classes;
  - interfaces;
  - components;
  - collaborations;
  - operations.
- **Uses:**
  - To model business processes and workflows.
  - To flowchart operations.
- A good activity diagram is focused on system's dynamic behavior.
- Activity diagrams used in many UP workflows.
- Activity diagrams are most commonly used in the following ways.
  - In the Analysis workflow**:**
    - To model the flow in a use case in a graphical way.